4、程序构建

嵌入式设备应用开发（程序构建）

原创

嵌入式-老费 已于 2023-08-19 12:34:17 修改

阅读量365 收藏

点赞数

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： linux 数据结构 运维

基于全志soc的linux嵌入式开发专栏收录该内容

73 篇文章¥49.90¥99.00

已订阅8折续费

[

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益 ›

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

摘要 嵌入式开发中编译环节至关重要。早期的makefile维护在大型项目中常见，但如今更多团队转向使用cmake简化工作。本文介绍了从命令行直接编译，编写makefile自动化，到使用cmake构建程序的过程，降低了编译脚本的复杂性和学习成本。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing@163.com】

编译是嵌入式开发很重要的一个环节。记得早年在上海一家通讯公司上班的时候，单位还专门有一个人维护编译脚本。当时用的是makefile，这位同学的主要工作就是替大家维护好各个项目的makefile。当然，因为编译的项目足够多，也足够复杂，所以才会专门安排一个人来专门负责这个事情。但对于个人或者是工作量不那么大的公司来说，找一个人负责makefile实在是没有必要。而且，目前除了命令行、makefile，很多公司都开始使用cmake来构建、编译程序，这样一样，编译脚本的工作量就大大减少了。

Makefile

1、使用命令行编译文件

```
#include <stdio.h>

int main(int argc, char* argv[])
{
    printf("this is just a demo!\n");
    return 0;
}
```

如果是c，那么选择gcc；如果是c++，那么选择g++即可。既然我们上面的代码demo.c，那么直接使用gcc编译即可，

```
gcc demo.c -g -o demo
```

2、使用makefile进行编译

使用命令行虽然比较直接，但是每次要输入那么多内容，也似乎很麻烦。所以，这个时候我们可以编写一个简单的Makefile来帮我们处理这个问题，

```
.PHONY: all clean

all: demo

demo: demo.o
    gcc demo.o -o demo

demo.o: demo.c
    gcc -c demo.c -o demo.o

clean:
    rm -rf demo.o demo
```

使用的话，其实也非常简单，只需要在makefile的同级目录下面输入make即可。这样你就会看到下面这样的打印，

```
$ make
gcc -c demo.c -o demo.o
gcc demo.o -o demo
```

如果中间的程序不需要了，这个时候直接输入make clean即可，

```
$make clean
rm -rf demo.o demo
```

3、使用cmake进行编译

很多人不喜欢makefile编译，倒不是makefile不好用，主要是makefile比较难写，里面的命令规则很多，完全是程序员思维。一段时间不用之后，过了不久重新来弄，还得再学一次，学习的成本很高。所以，这个时候，很多人就转到了cmake。本身用cmake编译也很方便，我们只需要一个这样的脚本CMakeLists.txt就可以了，

```
cmake_minimum_required (VERSION 2.8)
project (demo)
add_executable(demo demo.c)
```

创建好脚本之后，就可以mkdir build，进入build目录后，利用cmake生成makefile，

```
mkdir build
cd build
cmake ..
```

不出意外，可以看到这样的打印，

```
-- The C compiler identification is GNU 9.4.0
-- The CXX compiler identification is GNU 9.4.0
-- Check for working C compiler: /usr/bin/cc
-- Check for working C compiler: /usr/bin/cc -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Detecting C compile features
-- Detecting C compile features - done
-- Check for working CXX compiler: /usr/bin/c++
-- Check for working CXX compiler: /usr/bin/c++ -- works
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done
-- Generating done
-- Build files have been written to:
/home/feixiaoxing/Desktop/demo/build
```

需要注意，这里的步骤仅仅是帮助我们生成makefile。如果我们需要生成可执行文件的话，那么这个时候还需要我们自己输入make命令才可以，

```
$ make
Scanning dependencies of target demo
[ 50%] Building C object CMakeFiles/demo.dir/demo.c.o
```

```
[100%] Linking C executable demo
[100%] Built target demo
```

至此，我们就可以看到demo可执行文件被生成了。